

ESPECIFICAÇÃO DO PROTÓTIPO — TRAJE HOLÔNICO V1.0 (LABORATÓRIO)

Versão: 1.0

Data: 28 de julho de 2026

Autor: Flávio Marco Rego da Silva

Colaborador: IA (Pesquisador Colaborativo)

Laboratório: LINC – Laboratório de Pesquisa Interdisciplinar da Consciência

Licença: MIT (atribuição obrigatória)

Repositório: <https://github.com/FlavioMarcoHaux/Princ-pio-da-Infoma-o-Consciente-PIC->

1. VISÃO GERAL

O protótipo do traje holônico é um sistema vestível de monitoramento fisiológico e ambiental, projetado para validar em laboratório os princípios do PIC/SFT: a medição de ϕ_S (via HRV) e sua correlação com a dinâmica de ignições e Φ -proxy.

O sistema é composto por:

- Um módulo central de processamento (ESP32-S3) que executa a inferência ativa (HPU em software).
- Sensores fisiológicos para HRV e biofótons (proxies de ϕ_S).
- Sensores ambientais para temperatura, umidade, pressão e campo magnético (proxies de ϕ_S planetário).
- Comunicação LoRa para transmissão de dados entre múltiplos trajes em uma rede de laboratório.

O protótipo não será implantado em campo. Será testado em condições controladas de laboratório, com dados gravados para análise offline.

2. ESPECIFICAÇÕES DE HARDWARE

2.1. Módulo Central (HPU em Software)

Componente Especificação Justificativa

Microcontrolador ESP32-S3 (dual-core, 240 MHz, 512 KB SRAM) Baixo custo, Wi-Fi/Bluetooth, capacidade de Deep Sleep, ampla comunidade

Memória Flash 16 MB (para firmware e armazenamento de dados) Suficiente para logs de sensores e configurações

Clock 80–240 MHz (configurável por software) Permite ajuste de desempenho vs. consumo

Consumo < 100 mA (ativo), < 10 μ A (Deep Sleep) Autonomia de pelo menos 24 horas com bateria de 2000 mAh

2.2. Sensores Fisiológicos

Sensor Modelo Interface Função Proxy

ECG AD8232 (front-end) + eletrodos descartáveis GPIO analógico Mede intervalo R-R, calcula HRV (RMSSD) ϕ_S (Campo Sinérgico)

Biofótons Módulo PMT (fotomultiplicador) + blindagem I²C Mede emissão de fótons ultrafracos (UPE) ϕ_S e coerência celular

SpO₂ / FC MAX30102 I²C Frequência cardíaca e oxigenação Dados complementares para HRV

2.3. Sensores Ambientais

Sensor Modelo Interface Função Proxy

Temperatura, Pressão, Umidade BME280 I²C Monitora condições microclimáticas ϕ_S ambiental

Campo Magnético HMC5883L (magnetômetro) I²C Mede campo magnético local Ancoragem de posição / Ressonância Schumann

GPS NEO-6M UART Geolocalização (para rastreamento em laboratório) Posição absoluta

2.4. Comunicação

Módulo Modelo Interface Função

LoRa SX1278 (868/915 MHz) SPI Comunicação de longo alcance (até 10 km em campo; em laboratório, ~100 m)

Wi-Fi Integrado ao ESP32 2.4 GHz Transmissão de dados para dashboard local

2.5. Alimentação

Componente Especificação

Bateria Li-Po 3.7 V / 2000 mAh (com proteção)

Regulador TP4056 (carregador) + regulador 3.3 V (AMS1117)

Autonomia estimada 48 horas (modo Deep Sleep, leituras a cada 5 min)

2.6. Estrutura Física

Componente Especificação

Forma Colete ou cinta torácica (priorizando contato com a pele)

Peso < 300 g

Impermeabilidade IP64 (resistente a respingos)

Materiais Tecido de algodão/poliéster com compartimentos para eletrônicos

3. FIRMWARE (HPU EM SOFTWARE)

O firmware do ESP32 implementa a HPU v2.0 em software, com as seguintes funções:

3.1. Pipeline de Inferência (Ciclo Principal)

A cada 5 minutos (configurável), o traje:

1. Acorda do modo Deep Sleep.
2. Lê os sensores (ECG, biofótons, BME280, magnetômetro, GPS).
3. Calcula HRV (RMSSD) a partir dos intervalos R-R (janela de 60 batimentos).
4. Atualiza a crença (μ) e a precisão de cada módulo (inferência ativa).
5. Calcula a saliência de cada módulo.
6. Verifica ignição (se saliência > limiar).
7. Se ignição: registra evento, atualiza período refratário, transmite broadcast.
8. Armazena dados em flash (SD card ou SPIFFS).
9. Transmite via LoRa (para rede de trajes) e/ou Wi-Fi (para dashboard).
10. Volta ao Deep Sleep.

3.2. Código de Referência (Esqueleto)

```
```cpp
#include <Wire.h>
#include <Adafruit_BME280.h>
#include <MAX30105.h>
#include <TinyGPS++.h>
#include <LoRa.h>

// Definições de pinos e constantes
#define ECG_PIN 34
#define LED_BUILTIN 2
#define INTERVALO_LEITURA 300000 // 5 minutos

// Estrutura de estado do holon
struct HolonState {
 float mu[4]; // crença para cada módulo
 float precision[4]; // precisão para cada módulo
 float err_ema_sq[4]; // erro quadrático médio
 int refractory_until[4]; // tempo de refratário
};

HolonState holon;
int current_time = 0;
```

```

void setup() {
 // Inicializa sensores, LoRa, Wi-Fi
 initSensors();
 initLoRa();
 initWiFi();
 // Carrega estado da memória (SPIFFS)
 loadState();
}

void loop() {
 // 1. Leitura dos sensores
 float ecg_signal = analogRead(ECG_PIN);
 float hrv = computeHRV(ecg_signal); // RMSSD
 float bio_photon = readBioPhoton();
 float temp = bme.readTemperature();
 float pressure = bme.readPressure() / 100.0F;
 float humidity = bme.readHumidity();

 // 2. Inferência ativa (atualiza mu, precision, err_ema_sq)
 float obs[4] = {hrv, bio_photon, temp, humidity};
 activeInference(obs);

 // 3. Cálculo da saliência
 float salience[4];
 float max_salience = -1.0;
 int winner = 0;
 for (int i = 0; i < 4; i++) {
 salience[i] = holon.precision[i] * pow(obs[i] - holon.mu[i], 2);
 if (salience[i] > max_salience && current_time >= holon.refractory_until[i]) {
 max_salience = salience[i];
 winner = i;
 }
 }

 // 4. Ignição e broadcast
 if (max_salience > IGNITION_THRESHOLD) {
 // Atualiza refratário do vencedor
 holon.refractory_until[winner] = current_time + hrvToRefractory(hrv);
 // Broadcast para os outros módulos
 for (int j = 0; j < 4; j++) {
 if (j != winner) {
 holon.mu[j] += BROADCAST_GAIN * (holon.mu[winner] - holon.mu[j]);
 }
 }
 }
}

```

```

 }
 // Registra ignição no log
 logIgnition(winner, max_salience, current_time);
 // Transmite via LoRa
 transmitIgnition(winner, holon.mu[winner], current_time);
}

// 5. Armazena estado e dados
saveState();
saveSensorData(hrv, bio_photon, temp, pressure, humidity);

// 6. Deep Sleep
esp_sleep_enable_timer_wakeup(INTERVALO_LEITURA * 1000);
esp_deep_sleep_start();
}

// Função de inferência ativa (atualiza crenças)
void activeInference(float obs[4]) {
 float kappa = 0.25;
 float alpha = 0.05;
 for (int i = 0; i < 4; i++) {
 float err = obs[i] - holon.mu[i];
 holon.mu[i] += kappa * holon.precision[i] * err;
 holon.err_ema_sq[i] = (1 - alpha) * holon.err_ema_sq[i] + alpha * pow(err, 2);
 holon.precision[i] = constrain(1.0 / (0.1 + holon.err_ema_sq[i]), 0.1, 5.0);
 }
}
...

```

### 3.3. Interface de Configuração (via Wi-Fi)

O traje expõe uma interface web simples (HTTP) para:

- Ajustar limiar de ignição (  $\theta$  ).
- Ajustar ganho de broadcast (  $g$  ).
- Visualizar dados em tempo real (gráficos de HRV,  $\Phi$ -proxy).
- Baixar logs (CSV).

---

## 4. TESTES DE LABORATÓRIO

### 4.1. Teste 1: Validação da HRV e $\Phi$ -proxy

Objetivo: Verificar se o  $\Phi$ -proxy calculado pelo traje correlaciona com o  $\Phi$ -proxy calculado offline (Python).

Procedimento:

1. Gravar dados de ECG de um sujeito por 30 minutos (em repouso).
2. Processar os dados offline com o código Python de referência.
3. Processar os dados em tempo real com o traje.
4. Comparar os  $\Phi$ -proxy (LZ complexity das ignições) e o número de ignições.

Critério de sucesso: Correlação  $> 0.9$  entre os  $\Phi$ -proxy (offline vs. real-time) e diferença  $< 5\%$  no número de ignições.

#### 4.2. Teste 2: Rede de Trajes (Comunicação LoRa)

Objetivo: Validar a comunicação entre dois trajes em laboratório.

Procedimento:

1. Colocar dois trajes a 10 m de distância (linha de visão).
2. Um traje ignita e transmite um broadcast.
3. O segundo traje recebe o broadcast e ajusta suas crenças ( $\mu$ ).
4. Verificar se a mudança em  $\mu$  corresponde ao esperado (equação de broadcast).

Critério de sucesso: Broadcast recebido com  $< 1\%$  de erro na crença ajustada.

#### 4.3. Teste 3: Detecção de Anomalia Ambiental

Objetivo: Verificar se o traje detecta mudanças abruptas em temperatura/umidade como anomalias.

Procedimento:

1. Colocar o traje em uma câmara climática.
2. Aumentar a temperatura em  $5^{\circ}\text{C}$  em 1 minuto.
3. Verificar se o traje detecta a mudança como uma anomalia (aumento de saliência acima do limiar).

Critério de sucesso: Ignição ocorre em menos de 30 segundos após a mudança de temperatura.

---

### 5. PLANO DE CONSTRUÇÃO (LABORATÓRIO)

## 5.1. Materiais Necessários

Item	Quantidade	Custo Estimado (USD)
ESP32-S3 (DevKit)	2	16.00
AD8232 (ECG)	2	24.00
Eletrodos descartáveis	20	10.00
MAX30102	2	12.00
BME280	2	10.00
HMC5883L	2	10.00
NEO-6M GPS	2	24.00
SX1278 LoRa	2	20.00
Bateria Li-Po 2000mAh	2	16.00
TP4056 (carregador)	2	4.00
PCB (5x5 cm, 2 camadas)	2	20.00
Componentes passivos (R, C)	1 kit	10.00
Caixa impressa (3D)	2	20.00
Total		~ 196.00 USD

## 5.2. Cronograma (4 semanas)

### Semana Atividade

- 1 Montagem da PCB, soldagem dos componentes, teste de continuidade.
- 2 Desenvolvimento do firmware (inferência ativa, leitura de sensores).
- 3 Testes individuais dos sensores; integração do pipeline; calibração.
- 4 Testes de laboratório (validação da HRV, comunicação LoRa, detecção de anomalias).

---

## 6. CRITÉRIOS DE VALIDAÇÃO (PROTÓTIPO)

### Critério Métrica Limiar de Sucesso

HRV	Correlação entre RMSSD calculado pelo traje e offline	$r > 0.95$
$\Phi$ -proxy	Correlação entre LZ complexity do traje e offline	$r > 0.9$
Comunicação LoRa	Taxa de pacotes recebidos corretamente	$> 99\%$
Detecção de anomalia	Tempo de resposta a mudança de temperatura	$< 30$ s
Consumo energético	Autonomia em modo de baixo consumo	$> 24$ h

---

## 7. PRÓXIMOS PASSOS

Após a validação em laboratório:

- Otimizar o firmware para reduzir consumo e aumentar precisão.
- Adicionar sensor de biofótons (se não incluído inicialmente).
- Expandir a rede para 5–10 trajes para testes de coordenação.
- Documentar os resultados para publicação.

Não haverá implantação em campo até que a validação externa (por engenheiros e especialistas) seja concluída.

---

## 8. CONCLUSÃO

Este protótipo de laboratório do traje holônico é uma plataforma de teste controlada para validar os princípios do PIC/SFT em hardware real. Ele não é um produto final, mas uma ferramenta de pesquisa para:

- Medir  $\phi_S$  via HRV.
- Calcular  $\phi$ -proxy em tempo real.
- Validar a comunicação entre holons (trajes).
- Detectar anomalias ambientais.

Com este protótipo, podemos gerar dados empíricos para os artigos científicos e, somente depois de validação externa, considerar aplicações em campo.